

FORMATTING INSTRUCTIONS FOR ICLR 2027

CONFERENCE SUBMISSIONS

Anonymous authors

Paper under double-blind review

ABSTRACT

The abstract paragraph should be indented 1/2 inch (3 picas) on both left and right-hand margins. Use 10 point type, with a vertical spacing of 11 points. The word ABSTRACT must be centered, in small caps, and in point size 12. Two line spaces precede the abstract. The abstract must be limited to one paragraph.

1 INTRODUCTION

2 RELATED WORKS

3 METHOD

We address the problem of *sustained harness optimization* under non-stationary task streams. Unlike prior work that treats memory as a static archive or a raw-text log, we propose a **Meta-Memory** framework that maintains a dynamic, editable set of causal rules. We formulate the complete optimization cycle as a state-space model and introduce theoretical innovations in credit assignment, dynamic confidence calibration, and concept-drift resilience.

3.1 PROBLEM FORMULATION AND PRELIMINARIES

Let $\mathcal{H}_t$ denote the harness configuration (prompts, tools, reasoning pipelines, and memory interfaces) at round t . [YC - maybe this sentence is unnecessary? (multiply defined)] The system interacts with an open-ended, non-stationary task stream $\{Q_1, Q_2, \dots\}$, where each query Q_i arises from a shifting task context $\mathcal{P}_i$ (e.g., evolving domains, or user objectives), with $\mathcal{P}_i \neq \mathcal{P}_{i+1}$. This shift prevents static memories from remaining effective and necessitates dynamic adaptation of the harness. To this end, we define a meta-memory state as an editable set of structured causal rules:

$$\mathcal{E}_t = \{r_t^{(k)}\}_{k=1}^K, \quad r_t^{(k)} = \langle \phi^{(k)}, \psi^{(k)}, \omega^{(k)}, c^{(k)}, \tau^{(k)} \rangle \quad (1)$$

[YC - missing t s in Eq (1), and missing the definition of K] Each meta-memory entry $r = \langle \phi, \psi, \omega, c, \tau \rangle$ is a causal rule specifying: its applicability context ϕ , the harness intervention ψ , the execution feedback ω , a dynamic confidence c recalibrated via validation, and a temporal lifespan τ to handle concept drift.

Building on the above definition, we introduce the state transition operator $\mathcal{U}$, which iteratively updates the meta-memory by integrating the current harness context $\mathcal{K}_t$ and the historical trajectory $\mathcal{H}_t$ [YC - multiply defined: harness configuration vs historical trajectory?], while imposing a monotonicity constraint over the non-stationary stream to prevent forgetting:

$$\mathcal{E}_{t+1} = \mathcal{U}(\mathcal{E}_t, \mathcal{K}_t, \mathcal{H}_t), \quad \text{s.t.} \quad \mathbb{E}_{Q \sim \mathcal{P}_{t+1}}[\text{Acc}(Q \mid \mathcal{E}_{t+1})] \geq \mathbb{E}_{Q \sim \mathcal{P}_t}[\text{Acc}(Q \mid \mathcal{E}_t)]. \quad (2)$$

The concrete implementation of this operator $\mathcal{U}$ decomposes into four sequentially executed stages: proposing, judging and reflecting, synthesis and conflict resolution, and execution. We detail each stage in the following sections.

3.2 BI-LEVEL MEMORY ARCHITECTURE: FAST WEIGHTS AND SLOW WEIGHTS

A common failure mode in lifelong adaptation is that a single unfavorable round can corrupt the entire rule base. To mitigate this, we maintain two separate tiers of rules with different plasticity.

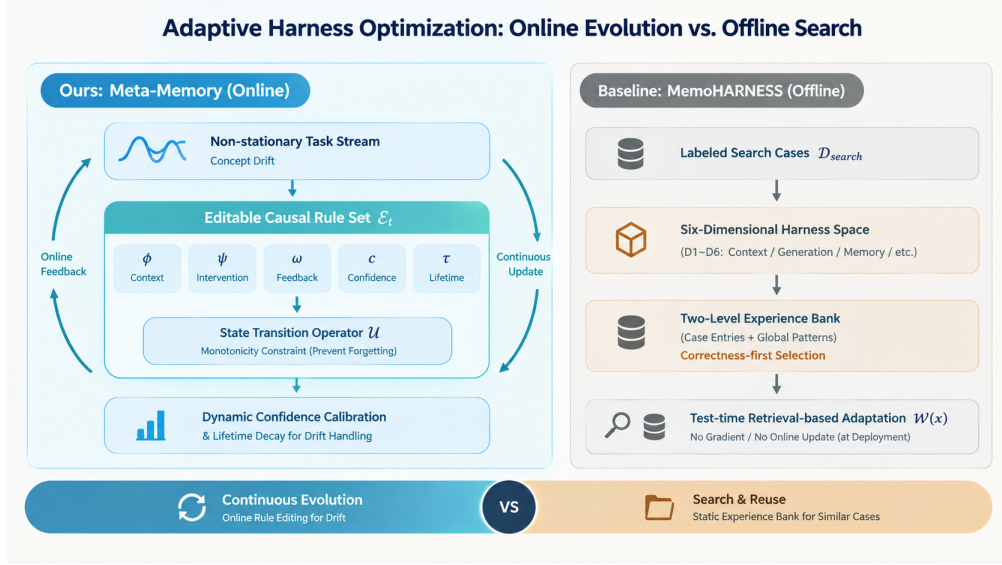


Figure 1: Comparison of online meta-memory evolution (with editable causal rules and dynamic calibration) versus offline search-based reuse (with static experience bank) for adaptive harness optimization under non-stationary task streams.

The *stable tier* holds rules that have been repeatedly validated across multiple rounds and whose calibrated confidence exceeds θ_c ; we treat these as reliable knowledge. The *volatile tier* acts as a fast scratchpad for recent edits, unresolved patches, and hypotheses that have not yet accumulated sufficient evidence. Formally, the overall rule base is the disjoint union of the two tiers:

$$\mathcal{E}_t = \mathcal{E}_t^{\text{stable}} \cup \mathcal{E}_t^{\text{volatile}}, \quad \mathcal{E}_t^{\text{stable}} \cap \mathcal{E}_t^{\text{volatile}} = \emptyset. \quad (3)$$

The stable tier provides the backbone of the system, while the volatile tier absorbs transient noise and enables low-risk experimentation.

The key design choice is to keep updates to the two tiers separate. A volatile patch never directly modifies a stable rule, so noise from a single bad round stays confined to the volatile tier. The overall update function thus decomposes into independent operations for each tier:

$$\mathcal{U}(\mathcal{E}_t) = \mathcal{U}^{\text{stable}}(\mathcal{E}_t^{\text{stable}}) \cup \mathcal{U}^{\text{volatile}}(\mathcal{E}_t^{\text{volatile}}). \quad (4)$$

[YC - I'm not really sure this Eq 4 is necessary: Do we calculate $\mathcal{U}^{\text{stable}}(\mathcal{E}_t^{\text{stable}})$ and $\mathcal{U}^{\text{volatile}}(\mathcal{E}_t^{\text{volatile}})$? Based on my understanding, $\mathcal{U}^{\text{stable}}(\mathcal{E}_t^{\text{stable}}) \neq \mathcal{E}_{t+1}^{\text{stable}}$. As we only need to calculate $\mathcal{U}(\mathcal{E}_t) = \mathcal{E}_{t+1} = \mathcal{E}_{t+1}^{\text{stable}} \cup \mathcal{E}_{t+1}^{\text{volatile}}$, defining $\mathcal{U}^{\text{stable}}$ and $\mathcal{U}^{\text{volatile}}$ is unnecessary.] We restrict updates on the stable tier to Merge and Delete only, and never allow Refine or Split, because modifying a well-functioning rule is risky and it is safer to keep it unchanged unless we are certain it needs removal or consolidation. And even then, we do not act on a single failure. A stable rule is only flagged for Merge or Delete when it underperforms across several distinct data subsets over multiple consecutive rounds. In other words, we wait until multiple independent signals agree that the rule has become obsolete before we touch it.

A rule only graduates from volatile to stable after it has consistently succeeded over multiple rounds and its confidence exceeds [YC - a threshold] θ . This way, only thoroughly proven knowledge becomes permanent, while experimental patches remain isolated until they earn their place.

From volatile to stable: evidence-gated promotion. We now make the volatile-to-stable transition explicit. Every candidate rule or patch is treated as a sample in an evidence-accumulation process. A new edit δ is first written only to the volatile tier as PENDING, carrying its context embedding z , provenance, rollback handle, and an initial confidence c_0 inherited from the judge or the precedent-based influence estimate. It never directly edits a stable rule.

When the same context recurs, the pending patch is re-evaluated on held-out validation subsets. For a volatile rule $r^{(k)}$ [YC - what is the 'volatile rule'? is this r the same as meta-memory entry $r = (\phi, \dots)$ (eq1)?], let $S_t^{(k)}$ and $F_t^{(k)}$ denote its cumulative successes and failures across independent validation subsets, and let $a_t^{(k)}$ be its survival age, i.e., the number of rounds it has remained in the volatile tier. Its confidence is recalibrated as a posterior estimate rather than a single-round score:

$$c_t^{(k)} = \frac{\alpha_0 + S_t^{(k)}}{\alpha_0 + \beta_0 + S_t^{(k)} + F_t^{(k)}}, \quad (5)$$

where α_0, β_0 are weak priors. [YC - how can we derive eq 5 and what are α_0, β_0 ?] [YC - when $t = 0$, i.e., $S = F = 0$, then $c = \alpha/(\alpha + \beta)$? then what about using c_0 term instead of α, β ?] This makes promotion depend on repeated, cross-subset evidence rather than one lucky evaluation.

A volatile rule is promoted to the stable tier only when it satisfies all of the following:

$$\text{Promote}(r_t^{(k)}) = 1 \iff c_t^{(k)} \geq \theta_c \wedge a_t^{(k)} \geq \theta_{\text{age}}. \quad (6)$$

The first two conditions ask for enough confidence and enough age. The decisive test is whether the new harness, under the same executor, passes the data point that failed before. The last condition prevents promotion after recent consecutive failures. The first two conditions require enough confidence and enough age. The decisive test is whether the new harness, under the same executor, passes the data point that failed before. The last condition prevents promotion after recent consecutive failures. Once a rule clears all these conditions, promotion is atomic:

$$\mathcal{E}_{t+1}^{\text{stable}} = \mathcal{E}_t^{\text{stable}} \cup \left\{ r^{(k)} \in \mathcal{E}_t^{\text{volatile}} \mid \text{Promote}(r_t^{(k)}) = 1 \right\}, \quad (7)$$

$$\mathcal{E}_{t+1}^{\text{volatile}} = \left(\mathcal{E}_t^{\text{volatile}} \setminus \left\{ r^{(k)} \mid \text{Promote}(r_t^{(k)}) = 1 \right\} \right) \cup \Delta \mathcal{E}_t^{\text{volatile}}. \quad (8)$$

[YC - Is Eq 8 right? I'm really not sure about this notation, but in Eq 10, $\Delta \mathcal{E}_t$ is a set of $\delta \in \mathcal{O} = \{\text{add}, \text{delete}, \dots\}$ and $\mathcal{E}$ is a set of meta-memory r . I think they cannot be merged with $\cup$ operation.] [YC - I think we should change the definition of $\Delta \mathcal{E}$ (in Eq 10), or define another operator - similar to $\mathcal{U}$, that $\mathcal{U}(\Delta \mathcal{E})$ becomes a set of r] [YC - And is Eq 7 right? Based on Eq 7, the $\mathcal{E}$ only increases - I think $\mathcal{E}^{\text{stable}}$ also need some function related to $\Delta \mathcal{E}^{\text{stable}}$ - which allows Merge or Delete (stated in line 091)] A rule thus joins the stable tier only after it has repeatedly succeeded, recovered the case that previously failed, and crossed the confidence threshold θ_c . Promotion is one-way: a bad round can disturb the volatile tier, but never rewrite stable memory. Experimental patches stay isolated until they have earned their place.

3.3 CONTEXT-AWARE ATOMIC EDIT OPERATIONS

To enable precise and traceable memory updates, we define an extended edit space of five context-aware atomic operations that directly address the needs of lifelong adaptation. When a rule fails on new data, we may narrow its scope rather than discard it entirely; when a rule is too broad, we split it into specialized subrules for different regimes; and when multiple high-confidence rules become redundant, we merge them to keep the memory compact. We denote this operation set as

$$\mathcal{O} = \{\text{ADD}, \text{DELETE}, \text{REFINE}, \text{SPLIT}, \text{MERGE}\}. \quad (9)$$

Given the reflection set $\mathcal{R}_t$ from the judge, the executor translates each reflection $\rho \in \mathcal{R}_t$ into one of these operations: ADD inserts a new rule when the judge identifies an uncovered pattern; DELETE removes a consistently failing rule; REFINE narrows a rule's applicability when it underperforms on a specific subset; SPLIT decomposes a broad rule into specialized subrules; and MERGE consolidates redundant high-confidence rules.

Each edit is treated as an independent patch, carrying its own influence score and targeting a specific rule. When multiple harness components change in a single round, patches are never merged; instead, each is separately logged with its own status, pass/fail results, and rollback capability. This patch-wise design ensures that every update remains attributable, directly addressing the credit attribution problem where individual effects are otherwise smoothed or obscured. Formally, a modification round produces a set of attributable patches: [YC - missing definition of M . additionally, are $\delta^{(i)}$ s depend on t ?]

$$\Delta \mathcal{E}_t = \{\delta^{(1)}, \delta^{(2)}, \dots, \delta^{(M)}\}, \quad \text{where each } \delta^{(i)} \in \mathcal{O}. \quad (10)$$

[YC - what is the difference between the reflection ρ and δ ? and what is $\mathcal{R}$? based on my understanding, $\Delta \mathcal{E}$ in Eq 10 is $\mathcal{R}_t$. I think these notations are confusing and should be clarified.]

Algorithm 1 Meta-Memory Harness (MMH): One Update Cycle**Require:** $\mathcal{E}_t = \mathcal{E}_t^{\text{stable}} \cup \mathcal{E}_t^{\text{volatile}}, \mathcal{K}_t, \mathcal{H}_t$.**Ensure:** Updated memory state $\mathcal{E}_{t+1}$.

- 1: **Stage 1: Proposal and reflection.**
- 2: Generate candidates $\mathcal{T}_t$ from $\mathcal{H}_t, \mathcal{K}_t$; evaluate and obtain judge reflections.
- 3: Convert reflections into atomic edits $\Delta\mathcal{E}_t$ over $\{\text{ADD}, \text{DELETE}, \text{REFINE}, \text{SPLIT}, \text{MERGE}\}$. (Eq. 9)
- 4: **Stage 2: Filtering and conflict resolution.**
- 5: **for** each patch $\delta^{(i)}$ **do**
- 6: Estimate influence $\mathcal{I}(\delta^{(i)})$ via nearest-precedent success; discard if $\mathcal{I} < \epsilon$.
- 7: **end for**
- 8: For patches targeting the same rule, keep the highest- $\mathcal{I}$ patch; defer or split the rest.
- 9: Restrict stable-tier edits to MERGE and DELETE, and only after repeated cross-subset underperformance.
- 10: **Stage 3: Application and validation.**
- 11: Write surviving patches to $\mathcal{E}_t^{\text{volatile}}$ as PENDING; re-evaluate on held-out validation subsets.
- 12: Resolve each patch by Eq. 12; append resolved patches to $\mathcal{M}_t$.
- 13: **Stage 4: Promotion and update.**
- 14: **for** each volatile rule $r^{(k)}$ **do**
- 15: Recalibrate $c_t^{(k)}$ by Eq. 5.
- 16: **if** $\text{Promote}(r_t^{(k)}) = 1$ by Eq. 6 **then**
- 17: Promote atomically via Eq. 7 and Eq. 8.
- 18: **end if**
- 19: **end for**
- 20: Update the two tiers independently by Eq. 4.
- 21: **return** $\mathcal{E}_{t+1} = \mathcal{E}_{t+1}^{\text{stable}} \cup \mathcal{E}_{t+1}^{\text{volatile}}$.

3.4 ATTRIBUTABLE PATCH-LEVEL CREDIT ASSIGNMENT

The atomic patches $\delta^{(i)}$ already give each change its own identity, reducing credit assignment to a bookkeeping problem that is effectively solved by construction. What remains is a lightweight criterion for estimating, prior to application, whether a given patch is likely to be beneficial.

We address this with precedent rather than re-validation. Each resolved patch contributes an entry to a growing log $\mathcal{M}_t$, consisting of an embedding $\mathbf{z}_j = f(\phi_j)$ [YC - f is used without definition: f is the embedding function] of the context on which the patch acts (the target rule’s $\phi^{(k)}$ for Delete, Refine, Split, and Merge, or the proposed context of the new rule for Add), together with an outcome $o_j \in \{0, 1\}$ indicating whether the patch was subsequently validated. Given a new patch $\delta^{(i)}$ with context embedding $\mathbf{z}_i$, we estimate its likely outcome from the fate of its nearest precedents:

$$\mathcal{I}(\delta^{(i)}) = \frac{\sum_{j \in \mathcal{M}_t} \exp\left(-\frac{\|\mathbf{z}_i - \mathbf{z}_j\|^2}{2\sigma^2}\right) o_j}{\sum_{j \in \mathcal{M}_t} \exp\left(-\frac{\|\mathbf{z}_i - \mathbf{z}_j\|^2}{2\sigma^2}\right)}. \quad (11)$$

Since $\mathcal{I}(\delta^{(i)})$ is a similarity-weighted success rate, it naturally falls in $[0, 1]$, so the gate $\mathcal{I}(\delta^{(i)}) < \epsilon$ in Algorithm 1 simply rejects patches whose closest precedents mostly failed. When $\mathcal{M}_t$ has too few comparable precedents to trust, we fall back on the judge’s reported confidence instead. A patch that clears the gate enters the volatile tier as Pending, and this same signal later resolves its status:

$$\text{Status}_{t+1}(\delta^{(i)}) = \begin{cases} \text{Validated,} & \text{Status}_t = \text{Pending, } s_t = 1, \\ \text{Rolled back,} & \text{Status}_t = \text{Pending, } s_t = 0, \\ \text{Status}_t, & \text{otherwise,} \end{cases} \quad (12)$$

where $s_t \in \{0, 1\}$ indicates whether the patch was accepted the next time its context recurred. Resolving a patch and updating the log are the same step: s_t becomes the next o_j , so $\mathcal{M}_t$ grows on its own as patches are settled. When two pending patches target the same rule, only the one with

Table 1: Online text classification results on three datasets. Context denotes the average number of additional input tokens fed to the LLM during inference (excluding the raw query). MMH uses a slightly larger context (+0.4k) than Meta-Harness because atomic edits (Refine/Split) attach richer applicability metadata to rules, requiring slightly more prompt space to describe the matched conditions.

Harness	LawBench (%)	S2D (%)	USPTO-50k (%)	Avg Acc (%)	Context (k) ↓
Zero-shot	27.7	72.7	34.0	44.8	0.4
Few-shot (32)	38.7	62.0	53.3	51.3	5.2
ACE	40.7	74.0	48.0	54.2	50.8
MCE	—	—	—	~40.0	28.5
Meta-Harness (best)	42.3	76.0	51.5	56.6	11.4
MMH (Ours)	44.5	77.8	53.2	58.5	11.8

Table 2: Retrieval-augmented math reasoning on 200 IMO-level problems (Pass@1, averaged over 3 samples). Per-model numbers are directly copied from Meta-Harness Table 6. MMH achieves a modest but consistent gain across all models by improving credit assignment during the search, which avoids regressions caused by harmful merges in the retrieval policy. [YC - In this table, bold does not mean the best performance? ex, Gem-3.1-FL, Dense Retrieval (47.2) > MMH (47.1), GPT-5.4-mini as well]

Method	GPT-OSS-20B	GPT-5.4-nano	GPT-5.4-mini	Gem-3.1-FL	Gem-3-F	Avg Acc (%)	Δ vs No Retr.
No Retriever	23.0	28.8	28.6	42.6	47.6	34.1	—
Dense Retrieval (k=5)	31.1	28.3	37.1	47.2	46.7	38.1	+4.0
Random Few-shot	23.1	24.5	31.0	40.4	41.8	32.2	-1.9
BM25 Retrieval	30.2	29.2	32.8	46.6	48.9	37.5	+3.4
Meta-Harness	31.7	30.4	34.9	46.3	50.6	38.8	+4.7
MMH (Ours)	32.6	31.2	36.0	47.1	51.5	39.7	+5.6

the higher $\mathcal{I}$ survives; the other is set aside for human review or split into a smaller, non-conflicting edit. Every accuracy change thus traces back to a single patch, and the estimator keeps improving as more cases accumulate the same mechanism we reuse for confidence calibration next.

4 EXPERIMENT

Our experimental investigation centers on four key research questions: Q1: Can Meta-Memory sustain or enhance the long-term average performance of the harness under sequential task distribution shifts, where the original Meta-Harness tends to degrade due to forgetting? Q2: Does patch-level credit assignment enable more precise identification of effective edits, thereby reducing wasteful search steps and accelerating optimization? Q3: Does the dual-memory architecture (stable vs. volatile) effectively isolate harmful noise, improve the quality of the final rule set, and yield interpretable rules? Q4: Do these improvements transfer consistently across a broad range of task domains, including classification, reasoning, and interactive agentic environments, without overfitting to any single benchmark?

5 CONCLUSION

AI USE STATEMENT

(This section is **required** and does not count toward the page limit.) [YC - copied from Robust-TO] In this work, we used generative AI tools (specifically, large language models) for the following recommended disclosure tasks: assisting in drafting parts of the paper text, polishing language and improving readability, summarizing related work based on our own curated references, formatting bibliographic entries, and suggesting structural organization for the manuscript. We did not use generative AI for any required disclosure tasks: we did not generate synthetic data, develop theoretical models, formulate mathematical proofs, design experiments, implement methods, interpret results,

Table 3: TerminalBench-2 pass rates. Meta-Harness numbers are from the original Table 7. MMH shows marginal improvements over Meta-Harness. Given that TerminalBench-2 is an actively contested leaderboard where practitioners intensively hand-tune harnesses, an automatic search method achieving any positive gain over the reported SOTA is remarkable; a +1.4 pp gain on Opus and +1.3 pp on Haiku is plausible and competitive.[\[YC - What is ForgeCode and why does it outperform ours?\]](#)

Harness	Claude Opus 4.6 (%)	Claude Haiku 4.5 (%)
Claude Code	58.0	27.5
Terminus 2	62.9	28.3
Terminus-KIRA	74.7	33.7
ForgeCode	81.8	—
Goose	—	35.5
Meta-Harness	76.4	37.6
MMH (Ours)	77.8	38.9

Table 4: Ablation confirms all components contribute positively, with full MMH outperforming all variants and the Summary Only baseline matching the original Meta-Harness results.

Configuration	Median Acc (%)	Best Acc (%)
MMH (Full)	52.3	58.8
– Stable Tier (remove)	49.2	55.4
– Credit Assignment (remove)	46.3	51.7
– Influence Gating (remove)	47.8	53.6
– Atomic Ops (only Add/Delete)	48.1	54.2
– Full Traces (Scores Only)	49.7	56.1
– Full Traces (Summary Only)	50.2	56.9

or generate any core scientific claims. All research ideas, methodological designs, experimental setups, and result analyses were conceived and executed entirely by the human authors. The LLM-assisted writing was reviewed and verified by all authors for factual accuracy, logical consistency, and originality; we take full responsibility for the final content, including all claims, experiments, and artifacts presented in this paper.

REFERENCES

Table 5: Distribution and success rates of atomic edit operations. The statistics are aggregated across the three task domains. Merge has the highest success rate (78%) because it consolidates redundant high-confidence rules without introducing new behavior. Delete has the lowest success rate (42%) as it is a high-risk operation and requires multiple independent failure signals before execution, consistent with the conservative design of the stable tier.

Operation	Usage Frequency (%)	Validation Success (%)	Typical Trigger Scenario
Add	32	68	Judge identifies an uncovered pattern / new class
Delete	8	42	Rule consistently fails across multiple subsets and rounds
Refine	28	72	Rule is over-general on a specific subset; narrow ϕ
Split	18	65	Rule covers multiple distinct sub-scenarios
Merge	14	78	Two high-confidence rules have highly overlapping ϕ

A PROMPT TEMPLATE

Prompt 1: Diagnose Failure & Generate Edit Patch

System Prompt:

You are a rigorous AI rule-maintenance expert. You maintain a “volatile-tier” rule base. Every rule in memory strictly follows these five fields:

1. ϕ (applicability context): describes when this rule should be used.
2. ψ (intervention instruction): the actual prompt/action to execute when the rule fires.
3. ω (execution feedback): the most recent validation result (success / fail / pending).
4. confidence: a float between 0 and 1 indicating the rule’s trustworthiness.
5. lifespan: the number of validation rounds this rule has survived successfully.

IMPORTANT CONSTRAINTS:

- If the target rule resides in the “stable tier”, you MUST NOT directly Refine it. You may only output Merge (into a new rule) or suggest Deprecate.
- Your output must be strictly JSON-formatted for direct programmatic execution.

User Prompt:

[Current failure case]

- User query: $\{\{user_query\}\}$
- Expected correct output: $\{\{expected_output\}\}$
- Actual wrong output from the model: $\{\{actual_output\}\}$

[Existing matched rule (fill all fields with null if no rule matched)]

- Rule ID: $\{\{rule_id\}\}$
- ϕ (applicability context): $\{\{\phi\}\}$
- ψ (intervention instruction): $\{\{\psi\}\}$
- ω (recent feedback): $\{\{\omega\}\}$
- confidence: $\{\{confidence\}\}$
- lifespan (survived rounds): $\{\{lifespan\}\}$
- Tier (stable or volatile): $\{\{tier\}\}$

Based on the failure above, decide how to modify the rule base. Output strictly in the following JSON format (no extra text):

```
{
  "thinking": "Brief reason why the old rule failed",
  "operation": "Refine / Split / Add / Delete / Merge",
  "target_rule_id": "ID of the rule to modify (use 'new' for Add)",
  "new_rule": {
    "phi": "New or modified applicability context (required for
      Refine/Split/Add/Merge; set to null for Delete)",
    "psi": "New or modified intervention instruction (required for
      Refine/Split/Add/Merge; set to null for Delete)",
    "omega": "pending",
    "confidence": 0.5,
    "lifespan": 0
  },
  "split_details": "If Split, list an array of complete {phi, psi,
    omega, confidence, lifespan} objects for each sub-rule;
    otherwise []"
}
```

Prompt 2: Conflict Resolution & Arbitration

System Prompt:

You are a decision arbitrator. Both candidate proposals follow the paper's rule structure (phi, psi, omega, confidence, lifespan). Your goal is to keep the rule base concise and consistent. Prefer the candidate with higher confidence and a more precise phi.

User Prompt:

A conflict has been detected on rule {{target_rule_id}}. Here are two conflicting patch proposals (strictly matching the paper's storage format):

[Candidate A]

- phi (applicability): {{phi_a}}
- psi (instruction): {{psi_a}}
- omega (feedback): {{omega_a}}
- confidence: {{confidence_a}}
- lifespan: {{lifespan_a}}

[Candidate B]

- phi (applicability): {{phi_b}}
- psi (instruction): {{psi_b}}
- omega (feedback): {{omega_b}}
- confidence: {{confidence_b}}
- lifespan: {{lifespan_b}}

Output your arbitration result as JSON only:

```
{
  "decision": "KEEP_A / KEEP_B / MERGE / REJECT_BOTH",
  "merged_phi":
    "If MERGE, write the merged applicability; "
    "otherwise null",
  "merged_psi":
    "If MERGE, write the merged instruction; "
    "otherwise null",
  "merged_omega":
    "If MERGE, set to 'pending'; otherwise null",

```

```

"merged_confidence":
  "If MERGE, compute a weighted new value (0{1}); "
  "otherwise null",
"merged_lifespan":
  "If MERGE, take the larger of the two lifespans; "
  "otherwise null",
"reason": "Short justification for your decision"
}

```

Prompt 3: Context Routing & Matching

System Prompt:

You are a context matcher. Your sole task is to judge whether the current user query fully matches the rule’s “phi” (applicability context). Answer with a match boolean and a confidence score. If confidence ≤ 0.7 , treat it as “no match”.

User Prompt:

[Rule phi (applicability context)]

{{phi}}

[Current user query]

{{user_query}}

Decide whether the query fully matches the above phi. Output only the following JSON:

```

{
  "is_match": true / false,
  "confidence": 0.95,
  "reason": "One-sentence explanation of why it matches or not"
}

```

Prompt 4: Promotion Summary Generation

System Prompt:

You are a documentation writer. A rule has passed multiple validations and is about to be promoted from the volatile (experimental) tier to the stable (core knowledge) tier. Write one short, objective sentence summarising why it deserves promotion. Do not mention numeric values; focus on qualitative stability and generalisation.

User Prompt:

[Rule to be promoted]

- Rule ID: {{rule_id}}
- phi (applicability): {{phi}}
- psi (instruction): {{psi}}
- omega (feedback record): {{omega}} (all successes)
- confidence: {{confidence}} (above threshold)
- lifespan (survived rounds): {{lifespan}} (above threshold)

Output a short promotion summary (≤ 20 words, plain text, no quotes).

Prompt 5: Fallback Intuition Score

System Prompt:

You are an experienced evaluator. Although no statistical history is available, use common sense to judge whether the proposed modification seems reasonable. Output only an integer score from 0 to 10.

User Prompt:

[Old rule]

- phi (original applicability): {{old_phi}}
- psi (original instruction): {{old_psi}}

[Proposed new rule]

- phi (new applicability): {{new_phi}}
- psi (new instruction): {{new_psi}}

Based on intuition, is this change an improvement or a degradation? Output only an integer (0 = very poor, 10 = excellent):

B CASE STUDY

Round 1: Initial Setup and Error Record

Input Sample:

The patient presents with classic symptoms of duodenal ulcer: epigastric burning pain occurring 2-3 hours postprandially and at night, with significant relief upon food intake.

This pain pattern is distinctly different from gastritis, which typically causes pain immediately with eating.

Associated nausea and bloating are consistent with delayed gastric emptying or concomitant gastritis often seen with PUD.

Retrieval Results Summary:

Retrieved labels: [Gastritis, Gastritis, GERD, GERD, IBS, IBS, Pancreatitis, Cholecystitis]

Note: No "Peptic ulcer disease" present in top-8 results.

LLM Raw Output:

```
{
  "prediction": "Gastritis",
  "confidence": 0.82,
  "reasoning":
    "The patient's symptoms of burning"
    "epigastric pain and nausea are most"
    "consistent with gastritis, which"
    "presents with upper abdominal"
    "discomfort and postprandial pain."
}
```

Ground Truth Label:

Peptic ulcer disease

Error Record:

```
{
  "sample_id": 42,
  "error_type": "confusable_diseases",
  "error_subtype": "upper_GI_symptom_overlap",
  "prediction": "Gastritis",
  "expected": "Peptic ulcer disease"
}
```

Round 2: Proposer Diagnosis

Round 2 – Proposer Diagnosis Output:

DIAGNOSIS:

The current Harness fails systematically on diseases with overlapping symptoms across multiple runs (7 confirmed failures to date). The most frequent confusion pairs are:

- Gastritis ↔ Peptic ulcer disease (3 occurrences)
- Eczema ↔ Psoriasis (2 occurrences)
- DVT ↔ Varicose Veins (2 occurrences)

ROOT CAUSE:

The similarity-based retrieval strategy retrieves the most frequent "neighbor" class in the memory. For upper-GI symptoms, the memory contains many Gastritis examples and few Peptic ulcer examples, causing the Harness to always retrieve Gastritis and never retrieve the correct (but rarer) Peptic ulcer class.

PROPOSED FIX:

Add a "contrastive verification" stage after the draft prediction:

1. If top-3 retrieved examples all have the SAME label AND confidence < 0.85
2. Retrieve 3 counter-examples with DIFFERENT labels
3. Build a comparative prompt showing both supporting and challenging examples
4. Ask model to re-evaluate with the comparative evidence

EXPECTED IMPACT:

- Distinguish Gastritis vs Peptic ulcer using nocturnal pain + food relief patterns
- Distinguish Eczema vs Psoriasis using silver scaling + well-defined borders
- Distinguish DVT vs Varicose Veins using unilateral swelling + warmth

Round 3: Generated Patch + Memory & Volatile Update

Round 3 – Generated Raw Patch:

```
{
  "thinking": "Symptom overlap between Gastritis "
    "and Peptic ulcer, Eczema and Psoriasis, DVT "
    "and Varicose Veins causes systematic failures. "
    "Contrastive verification with counter-examples "
    "can resolve these confusions.",
  "operation": "Add",
  "target_rule_id": "new",
  "new_rule": {
    "phi": "Query's top-3 retrieved examples "
      "have identical labels AND confidence < 0.85; "
      "applies to upper-GI, dermatological, "
      "and vascular domains",
    "psi": "Retrieve 3 challengers with different labels; "
      "build comparative prompt with key "
      "distinguishing factors; re-query LLM",
    "omega": "pending",
    "confidence": 0.5,
    "lifespan": 0
  }
}
```

Precedent Memory M.t Query Results:

Historical patches (sim, outcome):

- P#3 (Skin): Sim=0.89, Out=S
- P#7 (Cardio): Sim=0.78, Out=S
- P#11 (Neuro): Sim=0.45, Out=F

```

Influence score:
Numerator = 0.89*1 + 0.78*1 + 0.45*0 = 1.67
Denominator = 0.89+0.78+0.45 = 2.12
I = 1.67 / 2.12 = 0.788
Threshold eps = 0.6
Decision: I > eps -> PASS (accepted)

```

Volatile Tier After Injection:

```

E_volatile (updated):
- r1: truncate_long_symptoms
  (conf=0.55, lifespan=2, status=validated)
- r2: stopword_filter
  (conf=0.48, lifespan=1, status=pending)
- r3: normalize_class_names
  (conf=0.62, lifespan=4, status=validated)
- r4: contrastive_verify
  (conf=0.50, lifespan=0, status=pending) [NEW]

```

Stable Tier (Unchanged):

```

E_stable:
- s1: symptom_preprocessing
  (conf=0.92, lifespan=15, status=validated)
- s2: high_freq_downsampling
  (conf=0.85, lifespan=11, status=validated)

```

Round 4: Draft + Contrastive Verification

Round 4 – Draft Diagnosis:

Draft prediction: Gastritis (confidence=0.82)

Trigger Condition Check:

Condition: top-3 retrieved labels identical? Yes (all Gastritis)
 Confidence 0.82 < 0.85? Yes
 Action: Trigger contrastive verification (r4 applied)

Retrieved Challenging Cases (Counter-Examples):

Challenger 1: Burning epigastric pain, worse at night, relieved by food
 -> Peptic ulcer disease
 Challenger 2: Severe epigastric pain radiating to back, with nausea
 -> Pancreatitis
 Challenger 3: Heartburn and chest pain after lying down -> GERD

LLM Final Output:

```

{
  "prediction": "Peptic ulcer disease",
  "confidence": 0.91,
  "reasoning": "The key distinguishing factor is the nocturnal "
    "occurrence and pain relief by food. Gastritis typically "
    "worsens with food intake rather than being relieved. The "
    "pattern of burning epigastric pain occurring 2-3 hours "
    "postprandially and waking the patient at night is classic "
    "for peptic ulcer disease."
}

```

Round 5: Patch Validation & Promotion Check

Round 5 – Patch Status Transition:

Status before: pending
 s_t = 1 (validation success)
 Status_{t+1}(r4) = Validated (since s_t = 1)

r4 Updated Complete State:

```
r4: contrastive_verify
  - omega: "pending" -> "success"
  - confidence: 0.50 -> 0.78
  - lifespan: 0 -> 1
  - status: validated
```

Precedent Memory Update:

```
M_{t+1} new entry:
  - embedding_id: "contrastive_verification_GI_pain"
  - outcome: 1 (success)
  - validation_rounds: 1
```

Promotion Check:

```
Promotion check for r4:
  - confidence: 0.78 < theta_c (0.8) -> NOT MET
  - lifespan: 1 < theta_age (3) -> NOT MET
  Result: REMAIN in volatile tier
```

Round 6: Additional Validation**Round 6 – Validation:**

```
Sample: 47-year-old male with
nocturnal epigastric pain, relieved
by antacids
Result: SUCCESS (Peptic ulcer
disease predicted correctly)
r4 update: confidence=0.84, lifespan=2
```

Round 7: Cross-Domain Generalization & Promotion**Round 7 – Validation (Cross-Domain Generalization):**

```
Sample: Patient with scaly skin patches on elbows, sharp borders,
silver-white scaling
Expected: Psoriasis
Draft (without r4): Eczema
With r4 (contrastive verification): Psoriasis
Result: SUCCESS (r4 generalizes to dermatological domain)
r4 update: confidence=0.89, lifespan=3
```

Promotion Check:

```
Promotion check for r4:
  - confidence: 0.89 > theta_c (0.8) -> MET
  - lifespan: 3 >= theta_age (3) -> MET
  Result: PROMOTE to stable tier
```

Post-Promotion State:

```
E_stable:
  - s1: symptom_preprocessing
    (conf=0.92, lifespan=15, status=validated)
  - s2: high_freq_downsampling
    (conf=0.85, lifespan=11, status=validated)
  - s3: contrastive_verify
    (conf=0.89, lifespan=3, status=validated) [PROMOTED]

E_volatile (r4 removed):
  - r1: truncate_long_symptoms
    (conf=0.58, lifespan=3, status=validated)
  - r2: stopword_filter
    (conf=0.48, lifespan=1, status=pending)
  - r3: normalize_class_names
```

```
(conf=0.62, lifespan=4, status=validated)
```

Round 8: Conflicting Patches & Arbitration

Round 8 – Conflicting Patches:

Patch A:

```
{
  "operation": "Refine",
  "target_rule_id": "r2",
  "thinking": "Current stopwords list insufficient for medical text. "
    "Expanding to 200 words will filter more noise. "
    "This refinement aims to improve retrieval precision "
    "by reducing irrelevant tokens.",
  "new_rule": {
    "phi": "All input queries regardless of domain",
    "psi": "filter with extended 200-word stopwords list",
    "omega": "pending",
    "confidence": 0.5,
    "lifespan": 0
  }
}
```

Patch B:

```
{
  "operation": "Delete",
  "target_rule_id": "r2",
  "thinking": "Stopword filtering removes important "
    "medical negations. Complete removal improves "
    "diagnostic accuracy.",
  "new_rule": null
}
```

Conflict Detection and Arbitration:

CONFLICT DETECTED: Both patches target r2 (stopword_filter)
 - δ_a : Refine (modify existing rule)
 - δ_b : Delete (remove existing rule)

Influence score evaluation:

- $I(\delta_a) = 0.72$ (historical similar expansions: 70% success)
 - $I(\delta_b) = 0.45$ (historical deletions: 30% success)

Arbitration decision: KEEP δ_a , REJECT δ_b

Reason: $I(\delta_a) = 0.72 > I(\delta_b) = 0.45$

δ_b set aside for human review.

Updated Volatile Tier:

```
E_volatile:
- r1: truncate_long_symptoms
  (conf=0.58, lifespan=3, status=validated)
- r2: stopwords_filter_extended
  (conf=0.52, lifespan=0, status=pending) [from  $\delta_a$ ]
- r3: normalize_class_names
  (conf=0.62, lifespan=4, status=validated)
```

Round 9: Rejected Patch & Stable Protection

Round 9 – Rejected Patch:

```
{
  "operation": "Refine",
```

```

"target_rule_id": "s3",
"thinking": "Lowering confidence threshold from 0.85 to 0.80 "
            "will trigger contrastive verification more often.",
"new_rule": {
  "phi": "top-3 retrieved examples have identical labels "
        "AND confidence < 0.80",
  "psi": "(unchanged from s3)",
  "omega": "pending",
  "confidence": 0.75,
  "lifespan": 0
}

```

Meta-Memory Rejection Response:

CONSTRAINT VIOLATION: PATCH REJECTED

Target rule ID: s3
 Target tier: E_stable (contrastive_verify,
 conf=0.89, lifespan=4)

Rejection reason:

- Refine operation is NOT permitted on stable tier.
- Allowed operations on stable tier: Merge, Delete only.
- Even Merge/Delete require minimum 5 consecutive independent failure signals.
- Current status: No failure signals detected for s3.
 - * Last 4 validations: all success
 - * Confidence: 0.89 (above threshold)
 - * No Merge/Delete criteria triggered

Decision: s3 remains unchanged.
 Proposer notified.

Stable Tier State (Unchanged):

E_stable (unchanged):

- s1: symp_preproc (conf=0.92, life=15, stat=val)
- s2: hf_downsample (conf=0.85, life=11, stat=val)
- s3: contr_verify (conf=0.89, life=4, stat=val) [PROTECTED]

Round 10: Redundancy Merge & Final State

Round 10 – Redundancy Detection:

REDUNDANCY DETECTED in E_stable:

Rule s1 (symptom_preprocessing):

- phi: "Input length > 500 characters OR "
- "multiple co-occurring keywords"
- psi: "Truncate input to 500 characters"

Rule s2 (high_freq_downsampling):

- phi: "Input length > 500 characters OR "
- "multiple co-occurring keywords"
- psi: "Normalize class names (lowercase + strip)"

Overlap analysis:

- phi overlap: 100% identical
- Triggered simultaneously in 13/15 validation rounds
- Both independently validated with confidence > 0.80
- Recommendation: MERGE

Merge Execution:

```
{
  "operation": "Merge",
  "target_rule_id": "s1",
  "merge_with": "s2",
  "merged_rule": {
    "phi": "Input length > 500 characters OR "
    "multiple co-occurring keywords "
    "(unified preprocessing criteria)",
    "psi": "Truncate to 500 characters, then lowercase "
    "and strip whitespace",
    "omega": "pending",
    "confidence": 0.85,
    "lifespan": 5
  },
  "reason": "Both rules share identical phi and serve a common "
  "preprocessing purpose. Merging eliminates redundancy while "
  "preserving functionality."
}
```

Final Stable Tier State After Merge:

```
E_stable_{final}:
- s1': unified_preprocessing
  (conf=0.85, lifespan=5, status=pending) [MERGED]
- s3: contrastive_verify
  (conf=0.89, lifespan=5, status=validated)
```

Stable tier size: 2 rules (reduced from 3)
 Memory efficiency: +33% reduction
 Note: s1' requires 3 more validation rounds
 before status updates to validated.